TRIBHUVAN UNIVERSITY

INSTITUTE OF SCIENCE AND TECHNOLOGY

HIMALAYA COLLEGE OF ENGINEERING

CHYASAL,LALITPUR

Lab Report No:- 05

Title:- primality test, totient function and primitive root.

Submitted By:-

Name :- Anil Pal

Roll No:- 5

Submitted To:-

Department Of CSIT

Date Of Submission:- 2082/12/11

CamScanner

LAB5: primality test, totient function & primitive root.

Objective:
- To understand & implement methods for primality testing.
- To compute euler Totient function for given integer.
- To determine primitive roots of a number.
- To analyze their importance in cryptographic system.

Theory:

1. primality Testing
It is the process of determining wheather a given number is prime or not. A prime number is a number greater than 1 that has no divisors other than 1 & itself.

Methods:
- Trial division method:
check divisibility up to $\sqrt{n}$. Simple but inefficient for large numbers.
if p is prime number then:
$$a^{p-1} \equiv 1 \pmod{p}$$

- Miller-Rabin test:
A widely used probabilistic algorithm that is efficient for large numbers.

Importance:
prime numbers are fundamental in algorithms like RSA, Diffie-Hellman and ElGamal.

2. Eulers Totient function ($\phi(n)$)
The totient function counts the number of integers less than n that are relatively prime to n.

formula:
- if p is prime:
$$\phi(p) = p-1$$

- for general n:
$$\phi(n) = n\left(1-\frac{1}{p_1}\right)\left(1-\frac{1}{p_2}\right)\cdots\cdots$$

importance:

used in RSA algorithm for key generation.

$$e \cdot d \equiv 1 \pmod{\phi(n)}.$$

3.) Primitive Roots

A primitive root of a number n is an integer g such that its power generate all numbers from 1 to n-1 modulo n.

if g is a primitive root of prime P, then:

$$g^1, g^2, g^3, \ldots, g^{P-1} \pmod{p}$$

produces all non-zero residues.

Importance:

- used in Diffie-Hellman key exchange.
- used in ElGamal Encryption.

CamScanner

Q.1) program to find Euler Totient function value of any number

```
#include<stdio.h>
int gcd(int n1,int n2){
    int q,r1,r2,r;
    r1=n1;
    r2=n2;
  while(r2>0){
    q=r1/r2;
    r=r1%r2;
    r1=r2;
  }
  return r1;
}
int main(){
  int num,count,i;
  printf("Enter value of num to find totient function:");
  scanf("%d",&num);
  count=0;
  for(i=1;i<num;i++){
    if(gcd(num,i)==1){
      count++;
    }
  printf("\n Totient value of %d=%d\n",num,count);
  printf("Name: Anil Pal  Roll no: 5");
  return 0;
}
```

```
C:\Users\Dell\OneDrive\Desk
Enter value of num to find totient function: 15

Totient value of 15 = 8
Name:Anil Pal  Roll no:5
Process returned 0 (0x0)   execution time : 2.437 s
Press any key to continue.
```

Q.2) Given number is prime or not using Miller-Robin algorithm.

```
#include<stdio.h>
#include<stdlib.h>
#include<time.h>
long long mod_pow (long long a, long long b, long long n){
    long long result=1;
    a%=n;
while(b>0){
  if (b%2) result=(result*a)%n;
   a=(a*a)%n;
   b/=2;
  }
return result;
 }
int main(){
   long long n,d,a,x;
   int k=0;
  printf("Enter a positive integer:");
  scanf("%d",&n);
  if (n<=1){printf("%d is not prime\n",n);return 0;}
  if(n<=3){printf("%d is prime\n",n);return 0;}

  d=n-1;
  while(d%2==0){d/=2;k++;}
  printf("\n Step 1: Factor n-1\n");
  printf("n-1=2^k * d\n");
  printf("%d-1=2^%d * %d\n",n,k,d);

  srand(time(NULL));
  a=2+rand()%(n-3);
  printf("\n Step 2: Select random base a\n");
  printf(" Random a=%d\n",a);

  x=mod_pow(a,d,n);
  printf("\n Result: %d is prime\n",n);
  return 0;
  }
```

```
for(int i=0; i<k-1; i++)
{
  x=(x*x)%n;
  printf("Step %d; x=x^2 modn=%d\n",i+3,x);
if(x==n-1){
  printf("\n Result:%d is Prime;",n);
  printf("Name: Anil Pal Roll:5);
  return 0;
  }
}
printf("\n Result: %d Is not Prime\n",n);
 printf("Name: Anil Pal Roll no:5");
 return 0;
  }
```

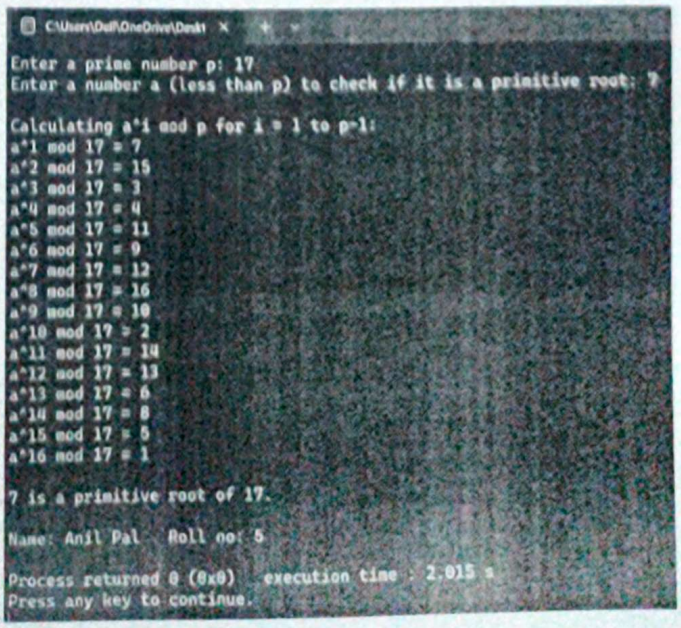
```
C:\Users\Dell\OneDrive\Desk1

Enter a positive integer: 17

Step 1: Factor n-1
n-1 = 2^k * d
17 - 1 = 2^4 * 1

Step 2: Select random base a
Random a = 5
Calculate x = a^d mod n
x = 5^1 mod 17 = 5
Step 3: x = x^2 mod n = 8
Step 4: x = x^2 mod n = 13
Step 5: x = x^2 mod n = 16

Result: 17 is prime
Name:Anil Pal   Roll no:5
Process returned 0 (0x0)   exec
Press any key to continue.
```

```
C:\Users\Dell\OneDrive\Desk1

Enter a positive integer: 15

Step 1: Factor n-1
n-1 = 2^k * d
15 - 1 = 2^1 * 7

Step 2: Select random base a
Random a = 7
Calculate x = a^d mod n
x = 7^7 mod 15 = 13

Result: 15 is not Prime
Name:Anil Pal   Roll no:5
Process returned 0 (0x0)   execut
Press any key to continue.
```

CamScanner

Q.3.) C Program to find primitive root of any prime number.

```
#include<stdio.h>
int power (int a, int i, int p){
    int result =1;
  for(int j=0; j<i; j++){
   result=(result * a) %p;
  }return result;
}
int main(){
  int a,p;
 printf("Enter a prime number p:");
 scanf("%d", &p);
 printf(" Enter a number a (less than p) to check if it is a primitive root:");
 scanf("%d", &a);
 int res[p-1];
 int isprimitive=1;
 printf("\n calculating a^i mod p for i=1 to p-1:\n");
  for(int i=1; i<p; i++){
    res[i-1]= power(a, i, p);
    printf(" a^%d mod %d=%d\n", i, p, res[i-1]);
    }
 if(!is primitive) break;
 }
 if(is primitive)
    printf("\n%d is a primitive root of %d.\n", a, p);
 else
  printf("\n %d is Not a primitive root", a, p);
 printf("Name: Anil Pal Roll no: 5");
 return 0;
}
```



DISCOSSION

In this lab, we explored three core mathematical concepts used in modern cryptography. Primality testing helps ensure that the numbers used in encryption are secure and difficult to factor. The totient function plays a crucial role in modular arithmetic & is especially important in RSA encryption. And, primitive roots are essential for cyclic group generation. Their role in discrete logarithm problems makes them vital for secure communication protocols.

CONCLUSION

In the lab, we were able to implement Miller Robin Algorithm, Totient function & primitive root in C programming.

CamScanner